



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών

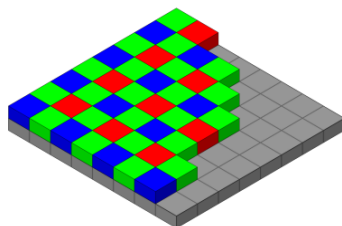
Εργαστήριο Μικροϋπολογιστών και Ψηφιακών Συστημάτων

Ψηφιακά Συστήματα VLSI

5η Εργαστηριακή Άσκηση

05/05/2026 (Αναπλήρωση)

Ομάδα 08



Ναταλία Λαγού (03122802)

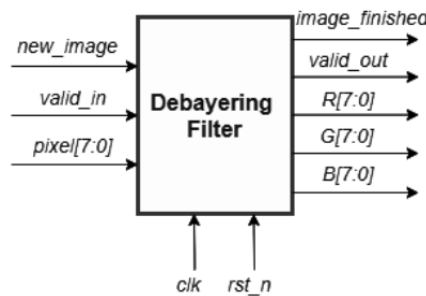
Διονύσιος Γεώργιος Μουρελάτος (03122055)

Υλοποίηση Debayering Φίλτρου με AXI διεπαφή σε Zynq SoC FPGA

Μέρος A

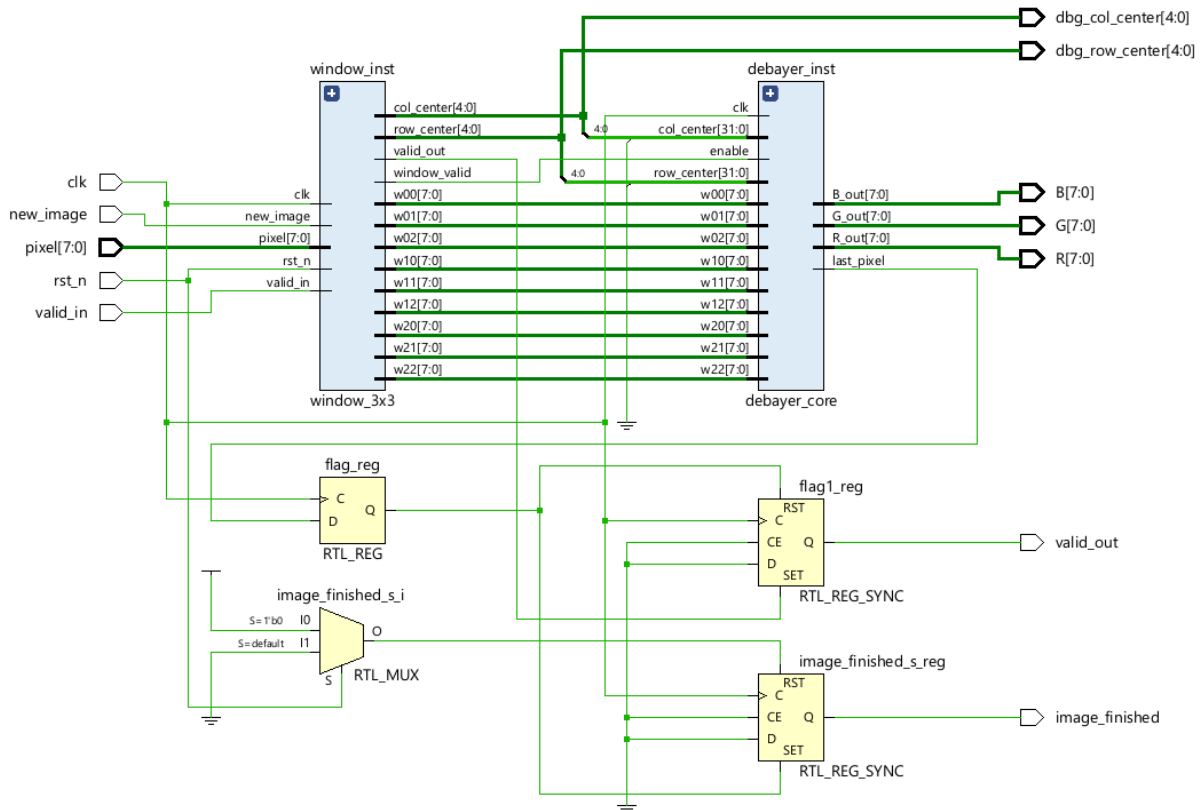
A.1 Ανάλυση Υλοποίησης

Στο Μέρος A της εργαστηριακής άσκησης υλοποιήσαμε το Debayering φίλτρο για επεξεργασία εικόνας και τη μετατροπή της σε RGB. Ακολουθήσαμε τους κανόνες και τα πρότυπα που ορίζονται στην εκφώνηση (GBRG mosaic, κλπ.), με σκοπό να πετύχουμε τελικά την εξής αρχιτεκτονική:



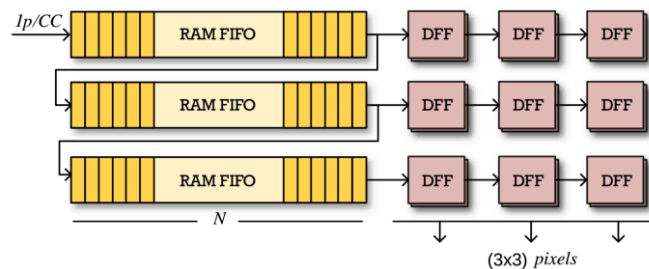
Elaborated Design Schematic

Το συνολικό κύκλωμα φαίνεται στο σχηματικό, το οποίο συμφωνεί με την επιθυμητή αρχιτεκτονική. Αποτελείται από δύο επιμέρους οντότητες, το **window_3x3** και το **debayer_core**, οι οποίες αναλύονται παρακάτω.



- **debayer_window.vhd**

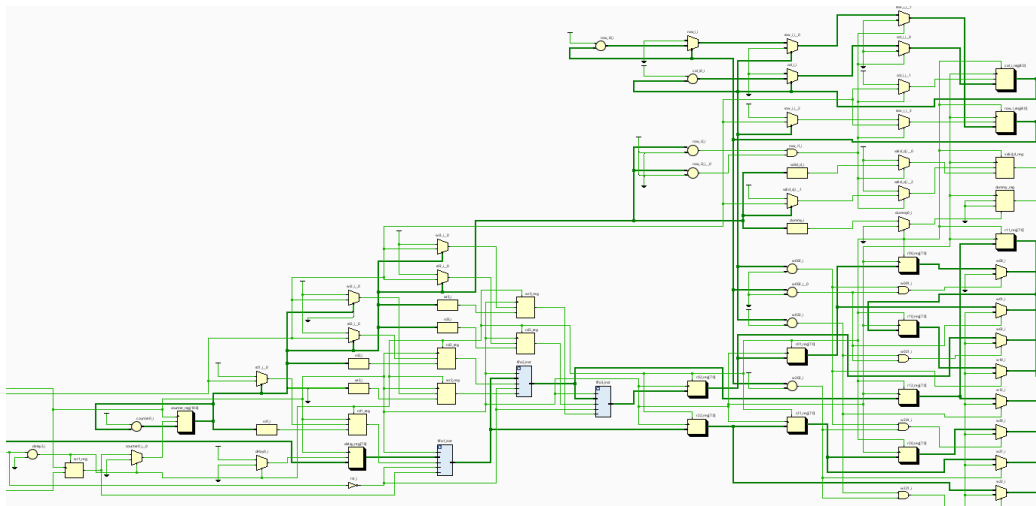
Στον κώδικα αυτόν ορίζεται το entity **window_3x3**, μία μοναδα ολισθαίνοντος παραθύρου, απαραίτητη για την επεξεργασία των γειτονικών pixels. Η αρχιτεκτονική βασίζεται σε έναν **S2P - Serial to Parallel Converter** υλοποιημένο με **3 Line Buffers (FIFO)** και **9 shift registers (DFFs)**, όπως φαίνεται στο σχήμα, ώστε να αποθηκεύει τα pixels εισόδου και να τροφοδοτεί την είσοδο του κυκλώματος υπολογισμού των μέσων όρων με όλα τα pixels της 3x3 γειτονιάς παράλληλα.



Για τις FIFO μνήμες χρησιμοποιήσαμε το έτοιμο IP της Xilinx από το Vivado επιλέγοντας βάθος N, όσο και το μέγεθος της εικόνας. Ως προς τον κώδικα:

- Μέσω ενός κεντρικού **counter**, ο κώδικας ελέγχει το χρονισμό της ανάγνωσης και εγγραφής στα FIFOs, εξασφαλίζοντας ότι το παράθυρο αρχίζει να παράγει έγκυρα δεδομένα μόνο αφού έχουν συμπληρωθεί οι απαραίτητες γραμμές καθυστέρησης (latency). Η έξοδος της κάθε FIFO συνδέεται με την είσοδο της επόμενης. Αρχικά, εγγράφουμε μόνο στην πρώτη FIFO (**wr1**), κι όταν αυτή γεμίσει, διαβάζουμε (**rd1**) και γράφουμε στη δεύτερη (**wr2**) κ.ο.κ.
- Περιλαμβάνεται λογική ελέγχου ορίων (boundary control) στις εξόδους **w00-w22**, η οποία μηδενίζει τις τιμές των γειτονικών pixels όταν το κεντρικό pixel βρίσκεται στις ακμές της εικόνας.
- Παρέχονται οι τρέχουσες συντεταγμένες (**row_center**, **col_center**) για τον προσδιορισμό της θέσης της επεξεργασίας στο πλέγμα NxN.

Schematic of "window_3x3"

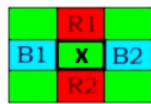


- **debayer_core.vhd**

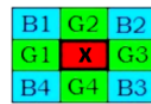
Ο κώδικας υλοποιεί τον υπολογιστικό πυρήνα του φίλτρου. Δέχεται ως είσοδο το παράθυρο 3x3 και ανάλογα με τη θέση του κεντρικού pixel στο πλέγμα (αν οι συντεταγμένες row_center και col_center είναι άρτιες ή περιττές), αναγνωρίζει τον τύπο του βασικού χρώματος (R, G ή B). Στη συνέχεια, υπολογίζει τις τιμές των χρωμάτων που λείπουν λαμβάνοντας τον μέσο όρο των γειτονικών pixel, ανάλογα με την κατηγορία της γειτονιάς (i-iv).



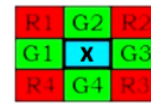
i



ii



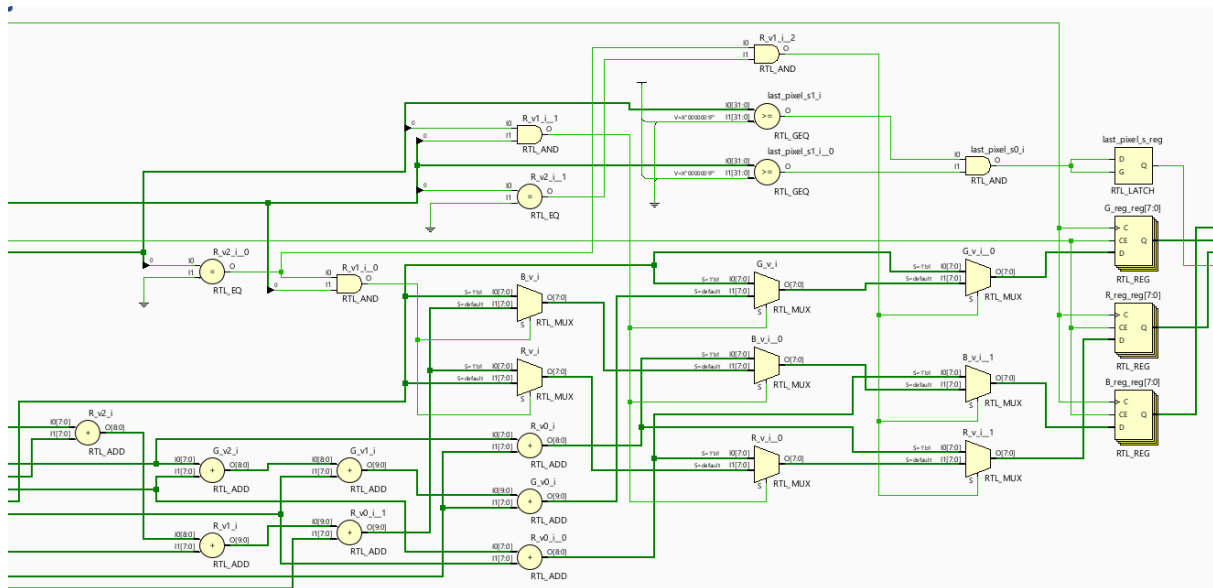
iii



iv

Για τη βελτιστοποίηση της ταχύτητας και την αποφυγή απωλειών ακρίβειας, οι υπολογισμοί γίνονται με χρήση της συνάρτησης resize (αποφυγή overflow), ενώ οι διαιρέσεις (average, /2 ή /4) υλοποιούνται μέσω δεξιάς ολίσθησης bit (shift right). Τέλος, ο πυρήνας περιλαμβάνει καταχωρητές εξόδου (**R_reg**, **G_reg**, **B_reg**) για τον συγχρονισμό των χρωματικών συνιστωσών, καθώς και ένα σήμα ελέγχου **last_pixel** για την επισήμανση της ολοκλήρωσης του πλαισίου της εικόνας.

Schematic of “debayer_core”



- **debayer_filter.vhd**

Ο κώδικας αυτός αποτελεί το **top-level entity** του συστήματος και διασυνδέει δομικά (structural architecture) τις δύο κύριες υπομονάδες που αναλύθηκαν προηγουμένως. Η οντότητα **debayer_filter** λαμβάνει τη σειρά των pixels της εικόνας και συντονίζει τη ροή των δεδομένων μεταξύ των βαθμίδων.

Συγκεκριμένα, η έξοδος του παραθύρου τροφοδοτεί απευθείας τον πυρήνα debayer, ενώ παράλληλα υλοποιείται ένας μηχανισμός ελέγχου για τα σήματα κατάστασης της εξόδου. Μέσω των βοηθητικών σημάτων **flag** και **flag1**, ο φίλτρο καθορίζει πότε οι έξοδοι RGB είναι έγκυρες (**valid_out**) και πότε ολοκληρώθηκε η επεξεργασία ολόκληρης της εικόνας (**image_finished**).

- **debayer_filter_tb.vhd**

Στο αρχείο αυτό υλοποιήθηκε το testbench για τον έλεγχο του φίλτρου.

A.2 Simulation, Latency & Throughput

Simulation Waveforms



Παραγόμενο αρχείο εξόδου:

```

input_N32x32.txt  output_sim.txt X
debayering_filter > debayering_filter.sim > sim_1 > behav > xsim > output_sim.txt
1  15 136 68
2  45 111 137
3  75 180 70
4  88 105 3
5  100 233 37
6  107 142 72
7  113 93 145
8  91 87 219
9  69 156 185
10 76 144 151
11 84 226 108
12 96 143 66
13 108 233 43
14 68 120 20
  
```

Python Script

Για την επιβεβαίωση των παραπάνω αποτελεσμάτων αναπτύξαμε κατάλληλο software (debayer.py) που προσομοιώνει τη λειτουργικότητα του φίλτρου.

Latency

delay <= pixel (Input pipeline)	→	+ 1 κύκλος
Πλήρωση 2 FIFO	→	+ 2N κύκλοι
Συμπλήρωση Παραθύρου 3x3	→	+ 2 κύκλοι
{R, G, B}_reg (Core registering)	→	+ 1 κύκλος
	⇒	2N + 4

Πράγματι, στην περίπτωση μας οι πρώτες τιμές εξόδου προκύπτουν στα 730 ns, ενώ το valid_in τίθεται στα 50 ns. Δεδομένου ότι η περίοδος του ρολογιού είναι 10 ns, προσδιορίζουμε το latency ως εξής:

$$latency = \frac{730 - 50}{10} = 68 \text{ κύκλοι} = 2 * 32 + 4$$

Οπότε η γενική σχέση επαληθεύεται για $N = 32$, όσο το μέγεθος της εικόνας.

Throughput

Σύμφωνα με τις αρχές του pipeline, μετά το αναπόφευκτο latency, λαμβάνουμε σωστή έξοδο σε κάθε παλμό ρολογιού, οπότε το **throughput = 1**.

A.3 FPGA Resource Utilization & Power Analysis

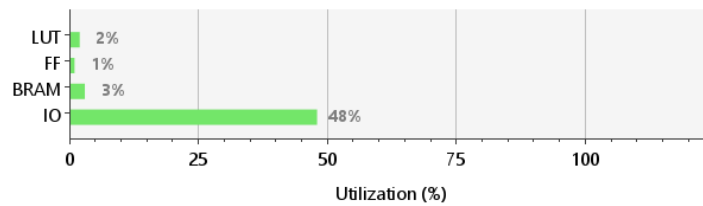
Resource Utilization Analysis

N = 32

Όσον αφορά την αξιοποίηση των φυσικών πόρων του FPGA, το design παρουσιάζει μια ισορροπημένη χρήση των διαθέσιμων στοιχείων του chip. Η αξιοποίηση των ακίδων εισόδου/εξόδου (I/O Utilization) ανέρχεται στο 48%, γεγονός που υποδηλώνει ότι το φίλτρο χρησιμοποιεί σχεδόν τις μισές διαθέσιμες ακίδες για τη μεταφορά των δεδομένων RGB, των σημάτων χρονισμού και των παραμέτρων ελέγχου.

Η χρήση των BRAM παραμένει σε εξαιρετικά χαμηλά επίπεδα (κάτω από 1%), καθώς οι Line Buffers που υλοποιήθηκαν μέσω των FIFOs καταλαμβάνουν ελάχιστους πόρους μνήμης για το μέγεθος εικόνας 32x32. Το ποσοστό αυτό καθιστά τη σχεδίαση ιδανική για ενσωμάτωση σε μεγαλύτερα συστήματα, καθώς αφήνει πάνω από το ήμισυ των πόρων του chip διαθέσιμο για επιπλέον λειτουργίες επεξεργασίας.

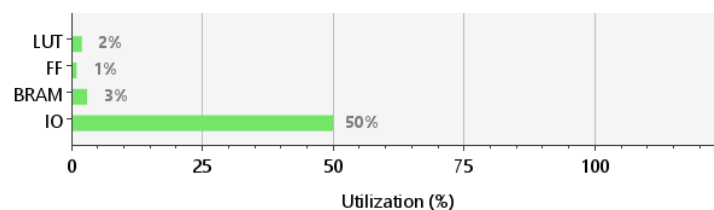
Resource	Utilization	Available	Utilization %
LUT	287	17600	1.63
FF	270	35200	0.77
BRAM	1.50	60	2.50
IO	48	100	48.00



Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ debayer_filter	287	270	112	287	1.5	48	1
debayer_inst (debayer_core)	14	25	23	14	0	0	0
> window_inst (window_3x3)	273	241	109	273	1.5	0	0

N = 64

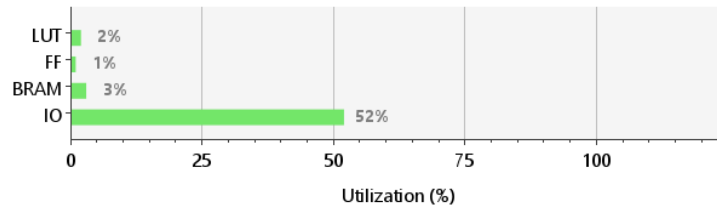
Resource	Utilization	Available	Utilization %
LUT	278	17600	1.58
FF	274	35200	0.78
BRAM	1.50	60	2.50
IO	50	100	50.00



Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ debayer_filter	278	274	108	278	1.5	50	1
debayer_inst (debayer_core)	14	25	21	14	0	0	0
> window_inst (window_3x3)	277	245	104	277	1.5	0	0

N = 128

Resource	Utilization	Available	Utilization %
LUT	274	17600	1.56
FF	278	35200	0.79
BRAM	1.50	60	2.50
IO	52	100	52.00



Name	1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N debayer_filter		274	278	118	274	1.5	52	1
▣ debayer_inst (debayer_core)		14	25	19	14	0	0	0
> ▣ window_inst (window_3x3)		273	249	114	273	1.5	0	0

Παρατηρούμε ότι η κατανάλωση πόρων παραμένει εξαιρετικά χαμηλή και σχεδόν αμετάβλητη και για τις διαφορετικές τιμές του N (LUT <2%, FF <1%, BRAM 2.5%), επιβεβαιώνοντας ότι η αρχιτεκτονική pipeline του debayer filter **κλιμακώνεται** αποδοτικά ανεξάρτητα από τη διάσταση της εικόνας.

Power Analysis

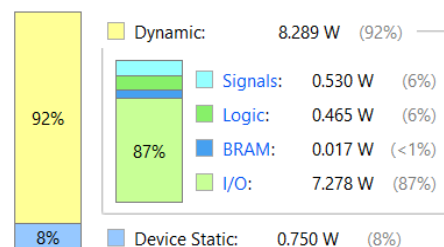
N = 32

Η αρχική εκτίμηση της κατανάλωσης ισχύος εμφάνισε μια αρκετά αυξημένη και πιθανώς μη ρεαλιστική τιμή, της τάξης των 8.521 W, με τη θερμοκρασία επαφής (Junction Temperature) να ανέρχεται στους 123.3°C, υπερβαίνοντας τα θερμικά όρια του chip. Η τιμή αυτή οφειλόταν κυρίως στην υπερεκτίμηση της δραστηριότητας των I/O, η οποία αντιπροσώπευε το 88% της δυναμικής ισχύος λόγω της έλλειψης δεδομένων προσομοίωσης (Low Confidence Level).

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 9.039 W (Junction temp exceeded!)
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 125.0°C
Thermal Margin: -44.3°C (-3.3 W)
Effective θ_{JA} : 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low

On-Chip Power

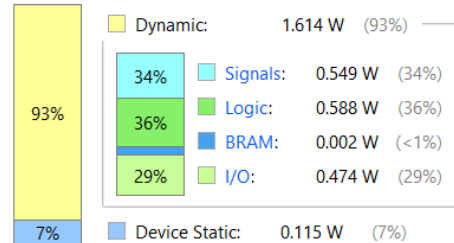


Με την εισαγωγή δεδομένων πραγματικής δραστηριότητας μέσω αρχείου SAIF, η συνολική ισχύς μειώθηκε στα 1.768 W και η θερμοκρασία σταθεροποιήθηκε στους 45.4°C. Δηλαδή επανήλθε εντός των ασφαλών ορίων λειτουργίας με θερμικό περιθώριο 39.6°C. Η τελική κατανομή δείχνει ότι η ισχύς των I/O περιορίστηκε στο 28% της δυναμικής κατανάλωσης, αποδεικνύοντας ότι το design είναι ενεργειακά αποδοτικό και θερμικά σταθερό.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.729 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 44.9°C
 Thermal Margin: 40.1°C (3.4 W)
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

On-Chip Power

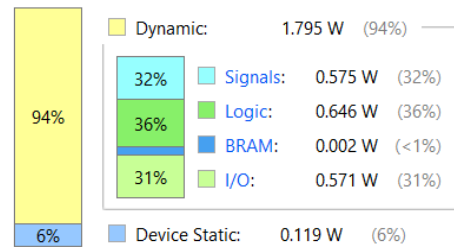


N = 64

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.914 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 47.1°C
 Thermal Margin: 37.9°C (3.2 W)
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

On-Chip Power

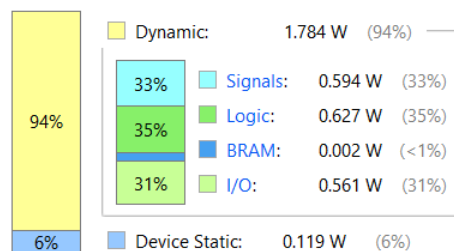


N = 128

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.902 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 46.9°C
 Thermal Margin: 38.1°C (3.2 W)
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

On-Chip Power



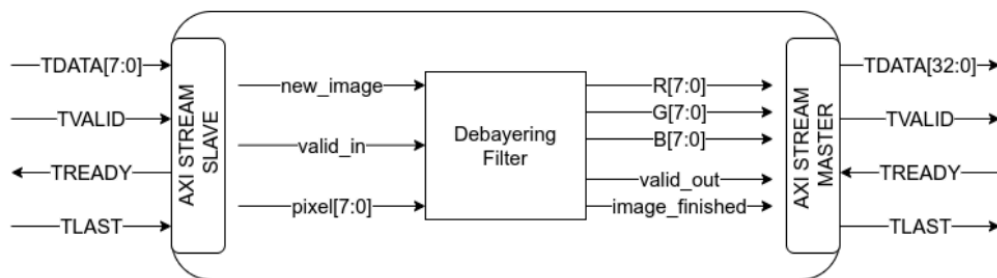
Και ως προς την κατανάλωση ισχύος, η υλοποίηση φαίνεται να είναι κλιμακώσιμη και αποδοτική. Η μείωση της, μάλιστα, για N=128, μπορεί να οφείλεται στο μειωμένο switching activity, αποτέλεσμα της επεξεργασίας μεγαλύτερης εικόνας σε συνεχή ροή.

Μέρος Β

Στο Μέρος Β της εργαστηριακής άσκησης, ζητούμενο ήταν ο προγραμματισμός της αναπτυξιακής πλακέτας ZYBO, ώστε να υλοποιεί το Debayering φίλτρο που σχεδιάστηκε στο Μέρος Α.

Τα δεδομένα εισόδου αποστέλλονται από τον ενσωματωμένο επεξεργαστή (ARM) προς το FPGA για επεξεργασία, και αντίστροφα, με DMA λογική. Επίσης, η επικοινωνία επεξεργαστή (PS) - FPGA (PL) βασίζεται στο πρωτόκολλο AXI.

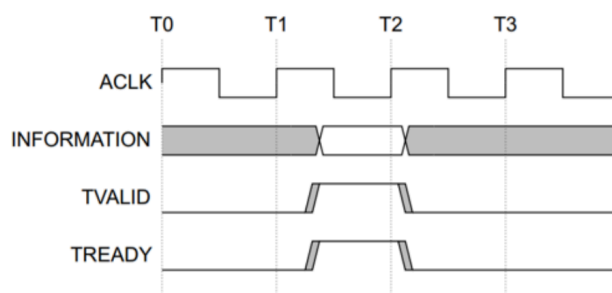
Η αρχιτεκτονική του Debayering φίλτρου με τη διεπαφή AXI4-Stream είναι η εξής:



Η διασύνδεση των σημάτων εισόδου και εξόδου του φίλτρου με τη διεπαφή AXI υλοποιήθηκε με τον ακόλουθο τρόπο:

```
Slave_TDATA[7:0]    = pixel[7:0]
Slave_TVALID        = valid_in
Master_TDATA[32:0]  = "00000000" & R[7:0] & G[7:0] & B[7:0]
Master_TVALID       = valid_out
Master_TLAST        = image_finished
```

Για την επικοινωνία των δύο DMA engines με τη διεπαφή AXI4-Stream, χρειάζεται να πραγματοποιηθεί το handshake μεταξύ T_VALID και T_READY:



Η οντότητα [dvlsi_lab6_top.vhd](#) αποτελεί την ενσωμάτωση του Zynq (PS) με τον επιταχυντή υλικού στο PL του FPGA. Συγκεκριμένα:

- Ο DMA του PS αποστέλλει τα pixels της εικόνας Bayer 8-bit προς τον επιταχυντή μέσω του καναλιού PS→PL (M_AXIS_TO_ACCELERATOR), θέτοντας το σήμα T_VALID σε λογικό '1' όταν τα δεδομένα είναι έγκυρα.

- Ο επιταχυντής (debayer_filter) αποδέχεται πάντοτε δεδομένα, καθώς το σήμα T_READY διατηρείται μόνιμα σε λογικό '1', αποφεύγοντας έτσι οποιαδήποτε καθυστέρηση στη ροή εισόδου.
- Για την ανίχνευση έναρξης νέας εικόνας, χρησιμοποιείται ένα κύκλωμα rising edge detector του σήματος tvalid_in: εάν στον τρέχοντα κύκλο ρολογιού το tvalid_in είναι '1' ενώ η καθυστερημένη του έκδοση (delay) είναι '0', τότε το σήμα new_image τίθεται σε '1' για έναν κύκλο, ενημερώνοντας τον επιταχυντή για την άφιξη νέας εικόνας.
- Στην αντίστροφη κατεύθυνση (PL→PS), ο επιταχυντής εξάγει τα επεξεργασμένα δεδομένα χρώματος (R, G, B) σε μορφή 32-bit (00RRGGBB) μέσω του καναλιού S_AXIS_S2MM_FROM_ACCELERATOR, συνοδεύοντάς τα με το σήμα tvalid_out, ενώ το τέλος κάθε εικόνας σηματοδοτείται μέσω του image_finished, το οποίο αντιστοιχίζεται στο σήμα tlast του AXI4-Stream.

Ο software κώδικας [helloworld.c](#) που εκτελείται στον επεξεργαστή ARM Cortex-A9 του Zynq αναλαμβάνει την αρχικοποίηση, τον έλεγχο και την επαλήθευση της επεξεργασίας debayering στο τμήμα PL.

- Αρχικά, απενεργοποιείται η κρυφή μνήμη δεδομένων (Xil_DCacheDisable) ώστε να διασφαλιστεί η συνοχή των δεδομένων μεταξύ PS και PL κατά τις μεταφορές DMA.
- Στη συνέχεια, αρχικοποιούνται δύο ανεξάρτητοι ελεγκτές DMA: ο TX_DMA (PS→PL) που αναλαμβάνει την αποστολή των pixels Bayer 8-bit από τη μνήμη DDR προς τον επιταχυντή, και ο RX_DMA (PL→PS) που λαμβάνει τα επεξεργασμένα δεδομένα RGB 32-bit (00RRGGBB) από τον επιταχυντή και τα αποθηκεύει στη μνήμη.
- Οι μεταφορές ξεκινούν πρώτα από το κανάλι λήψης (RX) και ακολούθως από το κανάλι αποστολής (TX), εξασφαλίζοντας ότι ο δέκτης είναι έτοιμος πριν ξεκινήσει η ροή δεδομένων.
- Η μέτρηση χρόνου εκτέλεσης του FPGA πραγματοποιείται μέσω του XTime_GetTime, καταγράφοντας τους κύκλους ρολογιού που μεσολαβούν από την έναρξη έως την ολοκλήρωση και των δύο μεταφορών DMA.
- Παράλληλα, εκτελείται μια αντίστοιχη υλοποίηση debayering σε λογισμικό αναφοράς απευθείας στον ARM.
- Τέλος, τα αποτελέσματα του FPGA συγκρίνονται pixel-by-pixel με αυτά του λογισμικού, υπολογίζεται το ποσοστό σφάλματος, και εκτυπώνεται αναφορά με τον αριθμό των mismatched pixels, καθώς και η επιτάχυνση (speedup) του FPGA έναντι της software υλοποίησης.

Η επιτάχυνση που παρατηρήσαμε ήταν περίπου **2500x** αποδεικνύοντας το σημαντικό πλεονέκτημα της υλοποίησης debayering σε FPGA μέσω pipeline αρχιτεκτονικής.